

Internship Report:
Distributed Infrastructure and Backend Engineering
at Juspay and Namma Yatri

A report submitted for end-term evaluation of the internship for the award
of
Bachelor of Technology
in
Information Technology

By
Raman Kapoor : 2022BIT-XXX

Under the Mentorship of
Pranshu Anand
(Industry Mentor, Juspay)

Faculty Advisor: **Dr. Vivek Kumar Singh**



ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
AND MANAGEMENT GWALIOR
GWALIOR, INDIA
2026

DECLARATION

I hereby certify that the work, which is being presented in the report entitled INTERNSHIP REPORT: DISTRIBUTED INFRASTRUCTURE AND BACKEND ENGINEERING AT JUSPAY AND NAMMA YATRI, in fulfillment of the requirement for end-term evaluation of the internship for the award of the degree of Bachelor of Technology in Information Technology and submitted to the institution is an authentic record of my own work carried out under the mentorship of Mr. Pranshu Anand (Juspay) and the academic guidance of Dr. Vivek Kumar Singh. I also cited the references about the text(s)/figure(s)/table(s) from where they have been taken.

Dated:

Signature of the candidate

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dated:

Signature of faculty advisor

Acknowledgements

I would like to express my sincere gratitude to my industry mentor at *Juspay*, Mr. Pranshu Anand, for the technical depth and the scope of ownership he extended to me during this internship. The work spanned distributed block storage, in-process caching for payment APIs, and a Shamir-secret-sharing-based secrets infrastructure — each a substantial engineering challenge in its own right — and his trust in letting me drive these projects end-to-end was the single biggest source of my growth this year.

I am equally grateful to the maintainers and contributors of *Namma Yatri*, the open-source ride-hailing platform on which I designed and shipped the distributed event-driven payout scheduler. The depth of the public review process, the rigour of the production constraints, and the willingness of the community to engage seriously with intern-led contributions were genuinely unusual and shaped how I now think about correctness in distributed systems.

I would also like to acknowledge my faculty advisor at ABV-IIITM Gwalior, Dr. Vivek Kumar Singh, for his continual support and supervision over the course of this internship and the parallel B.Tech project, and for the freedom he gave me to pursue ideas that crossed the boundary between industry practice and academic enquiry.

Finally, I thank my family and friends for their unwavering encouragement, and the faculty and staff of ABV-IIITM Gwalior for the academic environment that made this work possible.

Raman Kapoor

Abstract

This report presents the work undertaken across two production engineering engagements and one self-directed cloud-infrastructure project during the internship year. The first engagement, as a **Software Developer Intern at Juspay**, comprised three projects on the production payments platform: (i) replacing CEPH-based block storage with **Lightbits NVMe/TCP**, achieving $4\times$ storage efficiency by delivering 48,000 TPS over 4 nodes versus 37,000 TPS over 40 nodes while enabling active-active high-availability for Barbican via a shared backend; (ii) implementing a **thread-level and server-local caching** layer for high-traffic payment APIs that reduced cross-node network calls by over 4 % at peak and cut p95 latency by 11.2 %, while preserving correctness through formal cache-safety boundaries; and (iii) designing a **distributed secrets infrastructure** based on Shamir secret sharing, memory-only keys, and Raft-based replication, replacing heavy AWS KMS usage. The second engagement, as a **Backend Engineer (Open Source) at Namma Yatri**, delivered a distributed event-driven payout scheduling system that reduced driver payout latency from weekly cycles to T+2 hours under real production traffic of 10,000+ rides per day, backed by a Redis-based delayed job scheduler with per-ride locking and idempotent state management. The third workstream, a self-directed project on **PostgreSQL on Kubernetes/OpenStack**, architected a five-layer nested virtualisation stack on a single i9-14900 host, automated HA cluster provisioning end-to-end with Cluster API and Cloud-NativePG, and shipped a Zero-Trust secret-delivery framework using a Go-based OpenBao broker and a custom Python SDK. Together, these projects span block storage, in-process caching, distributed secrets, event-driven scheduling, and cloud automation — providing comprehensive exposure to production-grade distributed-systems engineering.

Contents

1	Introduction	1
1.1	Internship Overview	1
1.2	Themes	1
1.3	Report Structure	2
2	Block Storage with Lightbits	3
2.1	Introduction and Problem Statement	3
2.1.1	The Scaling Problem with Ceph	3
2.2	Lightbits: A New Block Storage Paradigm	4
2.2.1	Architectural Overview	4
2.2.2	Key Capabilities	5
2.2.3	Integration with Orchestration Platforms	5
2.3	Performance Benchmarking: Ceph vs. Lightbits	6
2.3.1	Test Configuration	6
2.3.2	Analysis of Results	6
2.3.3	Why the Difference Is So Large	6
2.4	Deployment Strategy	7
2.5	Summary	7
3	Distributed Caching for High-Traffic APIs	9
3.1	Context	9
3.2	Two-Tier In-Process Architecture	9
3.3	Cache-Safety Boundaries	10
3.4	Production Results	10
3.5	Relationship to the B.Tech Project	11
4	Distributed Secrets with OpenBao	12
4.1	Motivation	12
4.2	Shamir Secret Sharing for Unseal	12
4.3	Memory-Only Key Material	13
4.4	Raft-Based Storage Replication	13
4.5	Integration into the Application Path	14
4.6	Outcomes	14

5	Barbican KMS with HSM	15
5.1	Introduction and Motivation	15
5.2	Background Concepts	15
5.2.1	What Is Barbican?	15
5.2.2	What Is a Hardware Security Module (HSM)?	16
5.2.3	The Envelope Encryption Model	16
5.3	Architecture and Design	16
5.3.1	System Components	16
5.3.2	Per-Tenant Key Isolation	17
5.4	Implementation Details	18
5.4.1	HSM Configuration	18
5.4.2	Barbican Configuration	18
5.4.3	Secret Creation Flow	19
5.4.4	Secret Retrieval Flow	20
5.5	Security Properties	20
5.6	Challenges and Learnings	20
5.7	Summary	21
6	Distributed Payout Scheduling	22
6.1	Introduction and Business Context	22
6.2	System Requirements	22
6.3	Architecture Design	23
6.3.1	High-Level Architecture	23
6.3.2	Component Responsibilities	23
6.4	Detailed Design	24
6.4.1	Redis-Backed Delayed Job Scheduler	24
6.4.2	Payout Executor with Idempotent State Management	25
6.4.3	Paytm EDC Integration	26
6.5	Performance Characteristics	30
6.6	Challenges and Learnings	30
6.7	Summary	31
7	PostgreSQL on Kubernetes/OpenStack	32
7.1	Motivation and Scope	32
7.2	Five-Layer Nested Virtualisation Stack	32
7.2.1	Performance Tuning	33
7.3	Zero-Touch Cluster Provisioning	33
7.3.1	Time Breakdown	33
7.3.2	Failover Behaviour	34

7.4	Zero-Trust Secret Delivery Framework	34
7.4.1	Architecture	34
7.4.2	Custom Python SDK: osvault	35
7.5	Outcomes and Reusability	35
8	Conclusion	36
8.1	Summary	36
8.2	Skills Developed	36
8.3	Future Directions	37

List of Figures

5.1	Envelope Encryption Hierarchy in Barbican with HSM	17
6.1	High-Level Architecture of the Payout Scheduling System	23

List of Tables

2.1	Key Capabilities of Lightbits Labs NVMe/TCP Storage	5
5.1	Security Properties Achieved via Barbican and HSM Integration	21
6.1	Performance Characteristics of the Distributed Payout System	30

Chapter 1

Introduction and Overview

1.1 Internship Overview

This internship spanned two distinct industry engagements and one self-directed cloud-infrastructure project, each contributing to a different stratum of the modern distributed-systems stack.

- **Juspay** (Software Developer Intern, January 2026 – Present): production payments platform handling tens of thousands of transactions per second across the Indian fintech ecosystem. Three projects were delivered here, all on production traffic: a block-storage migration from CEPH to Lightbits NVMe/TCP, a two-tier in-process caching layer for high-traffic APIs, and a Shamir-secret-sharing-based replacement for AWS KMS.
- **Namma Yatri** (Backend Engineer, Open Source, November 2025 – January 2026): the open-source ride-hailing platform serving Indian cities, written in Haskell with PostgreSQL and Redis. Contributed a distributed, event-driven payout scheduling system for special zones, serving 10,000+ rides daily.
- **Self-directed** (Personal Infrastructure, March 2026): a five-layer nested virtualisation stack on a single workstation that exercises OpenStack, Kubernetes, CloudNativePG, and OpenBao end-to-end, automating HA Postgres provisioning and Zero-Trust secret delivery.

1.2 Themes

Three recurring themes connect these otherwise unrelated projects:

1. **Correctness under concurrency.** Whether the system in question is a payment API, a ride payout scheduler, or a multi-tenant secret store, the correctness boundary is the same: never duplicate a chargeable event, never lose a write, never serve stale data when authoritative state is required. This report repeatedly returns to the techniques that enforce that boundary — atomic operations, leases with fence tokens, idempotency keys, and formal cache-safety criteria.

2. **Doing more with fewer machines.** The Lightbits migration, the two-tier cache, and the OpenBao secret broker each ask the same question: *can we deliver the same or higher service level on a fraction of the infrastructure?* The answers, respectively, are 10× fewer storage nodes, 4 % fewer cross-node calls at peak, and a measurable reduction in AWS KMS cost.
3. **Open source as a delivery substrate.** Three of the five projects landed in publicly visible repositories — the Namma Yatri PR, the Lightbits Cinder driver integration, and the OpenBao broker — and the discipline of public review has been a recurring shaping force.

1.3 Report Structure

The remainder of this report is organised by project. Chapter 2 covers the Lightbits block storage migration. Chapter 3 describes the thread-level and server-local caching layer (the academic depth of which is the subject of the parallel B.Tech project report). Chapter 4 details the Shamir-secret-sharing-based secrets infrastructure. Chapter 5 covers the supporting Barbican KMS + HSM integration that preceded and motivated the OpenBao work. Chapter 6 presents the Namma Yatri payout scheduler. Chapter 7 describes the self-directed PostgreSQL on Kubernetes/OpenStack project. Chapter 8 concludes.

Chapter 2

High-Performance Block Storage: Lightbits as a CEPH Alternative

2.1 Introduction and Problem Statement

Modern cloud-native infrastructures—whether built on **OpenStack** for virtualisation or **Kubernetes** for container orchestration—require reliable, high-performance block storage as a foundational component. Backend database workloads, in particular, demand consistently high IOPS (Input/Output Operations Per Second) and predictably low latency to maintain transactional integrity and application responsiveness.

Many organisations initially adopt **Ceph**, an open-source, software-defined storage solution, to satisfy these requirements. Ceph’s distributed architecture, built on the CRUSH algorithm, provides redundancy and scalability through commodity hardware. While this approach is cost-effective for small-to-medium deployments, it introduces significant operational and performance challenges as the infrastructure scales to **hundreds or thousands of servers**.

2.1.1 The Scaling Problem with Ceph

At scale, Ceph’s inherent architectural characteristics create compounding difficulties:

1. **Disproportionate Hardware Consumption:** In large environments, Ceph’s storage cluster can consume up to **one-third of the total server estate**. This is because Ceph requires a significant number of OSD (Object Storage Daemon) nodes to distribute data, maintain replication factors (typically 3×), and handle recovery operations.
2. **Performance-Per-GB Demands:** High-performance workloads such as relational databases (PostgreSQL, MySQL) or time-series databases often require sustained throughput of **10–20 IOPS per GB**. For a single 5 TB volume, this translates to a demand of up to **100,000 IOPS**—a level that stresses even well-tuned Ceph clusters.
3. **Tuning Complexity:** As the cluster grows, the number of placement groups (PGs), OSDs, and MON/MGR daemons increases non-linearly. Optimal CRUSH map design, failure domain isolation, and network bandwidth planning become increasingly complex, requiring specialised expertise.

4. **Maintenance and Operational Overhead:** Larger clusters amplify maintenance challenges—OSD rebalancing during node failures can take hours, consuming network and CPU resources. Upgrades require careful rolling procedures, and monitoring systems must scale alongside the cluster.
5. **Tail Latency Issues:** Ceph’s consensus-based replication (using the RADOS protocol) introduces variable latencies, particularly under heavy load. Tail latency spikes are problematic for latency-sensitive database workloads where consistent sub-millisecond response times are critical.

Simply provisioning additional storage servers is a temporary and expensive mitigation. Each new node adds to the operational surface area without addressing the fundamental architectural limitations.

2.2 Lightbits: A New Block Storage Paradigm

Lightbits Labs offers an enterprise-class, software-defined block storage solution purpose-built for cloud-native environments. Unlike Ceph, which operates on a general-purpose distributed object store, Lightbits is architected from the ground up around the **NVMe-over-TCP (NVMe/TCP)** storage protocol—a protocol later ratified as an **industry standard** by the NVM Express[®] organisation.

2.2.1 Architectural Overview

The Lightbits architecture consists of the following key components:

- **LightOS:** The core storage operating system that manages NVMe SSDs, provides thin provisioning, snapshots, and data protection services.
- **NVMe/TCP Transport:** Eliminates the need for specialised RDMA networking hardware, operating over standard Ethernet infrastructure (25/100 GbE).
- **Elastic NVMe (eNVMe™):** A proprietary technology that provides hardware-level flash translation layer (FTL) optimisation, enabling efficient wear levelling, garbage collection, and I/O scheduling at the SSD firmware level.
- **Multi-Tenant Isolation:** Built-in quality-of-service (QoS) controls allow per-volume IOPS and bandwidth limits, ensuring tenant isolation in shared infrastructure.

Feature	Description
Cluster Scale	Up to 10 million IOPS from a minimal 4-node cluster
Protocol	NVMe-over-TCP (no RDMA/RoCE required)
Data Protection	Erasure coding, synchronous replication, snapshots
Thin Provisioning	Logical volumes over-provisioned without pre-allocating physical storage
Compression	Inline, transparent data compression reducing effective capacity cost
API-Driven	RESTful API, OpenStack Cinder driver, Kubernetes CSI driver

Table 2.1: Key Capabilities of Lightbits Labs NVMe/TCP Storage

2.2.2 Key Capabilities

2.2.3 Integration with Orchestration Platforms

Lightbits provides first-party, in-house-developed drivers for seamless integration:

OpenStack (Cinder Driver): Registers as a new `volume_type` in the Cinder block storage service. Once configured, tenants can provision Lightbits-backed volumes using the same OpenStack Horizon dashboard and CLI workflows they already use. The driver exposes enterprise features such as:

- Volume snapshots and clones
- QoS policy enforcement
- Live volume migration between Lightbits nodes
- Multi-attach support for shared-nothing database clusters

Kubernetes (CSI Driver): Implements the Container Storage Interface specification. After creating a new `StorageClass` referencing the Lightbits CSI provisioner, Kubernetes administrators and developers can request Persistent Volumes (PVs/PVCs) as they normally would. Features include:

- Dynamic provisioning
- Volume expansion (online resize)
- Snapshot and restore via `VolumeSnapshot` CRDs
- Topology-aware scheduling

2.3 Performance Benchmarking: Ceph vs. Lightbits

To quantify the impact, a head-to-head benchmark was conducted in a real customer environment running a **PostgreSQL** database workload. The metric of interest was **Transactions Per Second (TPS)**, measured using the standard `pgbench` benchmarking tool under sustained mixed read/write workloads.

2.3.1 Test Configuration

Parameter	Ceph Cluster	Lightbits Cluster
Number of Storage Nodes	40	4
NVMe SSDs per Node	24	9
Total NVMe SSDs	960	36
Replication Factor	3×	Erasure Coded
Network	25 GbE	25 GbE
Result: TPS	37,000	48,000

2.3.2 Analysis of Results

- **Performance Gain:** Lightbits delivered **29.7% higher TPS** (48,000 vs. 37,000) despite using dramatically fewer resources.
- **Hardware Reduction:** The storage node count was reduced by **90%** (40 → 4 nodes), and the total SSD count was reduced by **96.25%** (960 → 36 drives).
- **TCO Implications:** Beyond hardware savings, the reduction in nodes translates to lower power consumption, reduced rack space, fewer network ports, and significantly reduced operational overhead (fewer nodes to monitor, patch, and maintain).
- **Scalability Headroom:** The customer’s existing 40-node infrastructure, when repurposed with Lightbits, would provide an order-of-magnitude increase in available storage performance, enabling workload consolidation and future growth without additional hardware procurement.

2.3.3 Why the Difference Is So Large

The performance differential is attributable to several architectural factors:

1. **Protocol Efficiency:** NVMe/TCP has significantly lower protocol overhead compared to Ceph’s RADOS protocol, which involves multiple software layers (librados → OSD → BlueStore → kernel block device).

2. **Reduced Replication Overhead:** Lightbits uses erasure coding with configurable protection levels, requiring less raw capacity and fewer I/O amplifications compared to Ceph's default $3\times$ replication.
3. **Flash-Optimised I/O Path:** Lightbits' eNVMe technology bypasses the generic Linux block layer, providing a direct, optimised I/O path from the application to the NVMe SSD.
4. **Consistent Low Latency:** The absence of consensus-based I/O acknowledgement (as in Ceph's PG peering) eliminates a significant source of latency variability.

2.4 Deployment Strategy

The deployment of Lightbits was designed to be minimally disruptive:

1. **Infrastructure Assessment:** Audit existing Ceph hardware to identify 4 nodes suitable for Lightbits installation (sufficient NVMe slots, adequate network bandwidth).
2. **LightOS Installation:** Deploy LightOS on the selected nodes. The installation is a standard Linux-based process with automated provisioning scripts.
3. **Driver Integration:**
 - For OpenStack: Install the Cinder driver, create a new `volume_type`, and update the Cinder configuration (`cinder.conf`).
 - For Kubernetes: Deploy the CSI driver via Helm chart, create the `StorageClass` manifest.
4. **Workload Migration:** Migrate target workloads (e.g., PostgreSQL databases) to Lightbits-backed volumes using live migration (OpenStack) or PVC cloning (Kubernetes).
5. **Validation and Tuning:** Run benchmark suites to validate performance targets, configure QoS policies per volume, and set up monitoring dashboards.
6. **Decommission Surplus Ceph Nodes:** After successful migration and validation, the freed Ceph nodes are repurposed or decommissioned.

2.5 Summary

This project demonstrated that Lightbits provides a transformative alternative to Ceph for high-performance block storage workloads. By consolidating storage from 40 Ceph nodes

down to 4 Lightbits nodes while simultaneously improving database throughput by nearly 30%, the solution delivers compelling total cost of ownership savings and operational simplification. The tagline encapsulates the value proposition:

“Give us 4 of your CEPH nodes and let us solve your block storage performance challenges!”

Chapter 3

Thread-Level and Server-Local Caching for High-Traffic Payment APIs

3.1 Context

The payment APIs operated by Juspay process millions of transactions per day across a horizontally scaled fleet of stateless application nodes. Profiling on the production cluster showed that each request issues between 5 and 15 *cross-node network calls* during its lifecycle — typically to a Redis cluster for merchant configuration, gateway routing tables, feature flags and short-lived transactional state. A non-trivial fraction of these calls are *redundant*: identical lookups occur within the same request, or within milliseconds across concurrent requests on the same node, fetching effectively immutable data. The cumulative network round-trip time amplifies p95 and p99 tail latency, which is the metric customers actually feel.

The challenge in payment systems is that aggressive caching must not weaken correctness. Stale reads in this domain are not merely a UX issue — they can cause double-charges, lost merchant configuration updates, or compliance violations. The project therefore designed a caching layer governed by formal *cache-safety boundaries*, so that only data classes that provably satisfy strict immutability and idempotency criteria are eligible for caching.

3.2 Two-Tier In-Process Architecture

The caching layer is entirely in-process; no new infrastructure is introduced. Two tiers are layered:

Tier 1 — Thread-Level Cache. A per-request `IORef HashMap` that memoises lookups for the lifetime of a single request and is destroyed at request termination. Because the cache is not shared between threads, it cannot serve stale data — any value it returns is one that the same request has already observed. This tier alone achieved a 22.3 % hit rate on eligible lookups.

Tier 2 — Server-Local Cache. A node-scoped, concurrent cache implemented as a `TVar` containing a bounded LRU map with TTL-based expiry. TTLs are tuned per data class

(30 s for routing tables, 60 s for feature flags, 120 s for merchant configuration). This tier is shared across all request-handling threads on a node and achieved a 41.7 % hit rate on cross-request lookups.

3.3 Cache-Safety Boundaries

To prevent the deployment of caching on data that should not be cached, four formal criteria were defined. A data class is cache-eligible *if and only if* all four hold:

1. **Immutability within TTL** — the data must not change over the cache TTL window with probability greater than an agreed bound.
2. **Request-scope independence** — the value depends only on global or slow-changing state, not per-request parameters.
3. **Semantic idempotency of stale reads** — even a stale value, if served, does not produce a financial discrepancy.
4. **Deterministic cache key** — the key is derivable deterministically from the request, with no collision risk.

Single-resource state (inventory counters, balances, unique tokens) fails criterion (3) by construction and is therefore explicitly excluded from this caching layer; such state is handled by a separate contention layer, the design of which is the subject of the parallel B.Tech project report.

3.4 Production Results

The architecture was rolled out in stages on a cluster of ~ 24 application nodes serving roughly 68,000 cross-node operations per second at peak. The following improvements were measured over a 30-day post-deployment window:

- Combined Tier-1 + Tier-2 hit rate: **54.8 %** on eligible lookups.
- Cross-node calls per second eliminated: **$\sim 2,800$** at peak.
- Total cross-node call reduction at peak: **4.1 %**.
- p95 API latency: **98 ms $\rightarrow$ 87 ms (–11.2 %)**.
- p99 API latency: **285 ms $\rightarrow$ 248 ms (–13.0 %)**.
- Redis p99 latency: **12.5 ms $\rightarrow$ 9.2 ms (–26.2 %)**, driven by the removed load on the shared cluster.

- Memory overhead: under 20 MB per node.
- Cache-related correctness incidents: **zero**.

3.5 Relationship to the B.Tech Project

The academic treatment of this work — a deeper formalisation of the cache-safety boundary framework, an extension that handles single-resource contention via distributed locking, atomic Lua decrement, OCC, PN-Counter CRDTs and single-flight request coalescing, and an evaluation against examiner challenges raised at the mid-evaluation — is presented in the parallel B.Tech research-paper report. This chapter restricts itself to the *deliverable engineering* on production traffic at Juspay; the academic contribution is referenced for completeness but not duplicated.

Chapter 4

Distributed Secrets Infrastructure with Shamir Secret Sharing and Raft

4.1 Motivation

The pre-existing secrets infrastructure relied heavily on AWS KMS for envelope encryption of customer-scoped secrets. While correct, this design had three operational pain points that motivated a redesign:

1. **Cost coupling to traffic.** Every Decrypt on a hot path was a billable AWS API call. Under burst traffic the KMS bill scaled superlinearly and could not be amortised across requests without weakening the security model.
2. **Single trust anchor.** The KMS root key, although protected by AWS's HSM-backed CMK, represented a single external trust anchor whose compromise (or whose access-policy misconfiguration) would unseal every customer's secrets. Several auditors and large merchant customers had begun asking explicitly for a self-custodial, on-premises key-distribution mechanism with no single-key compromise path.
3. **Operational coupling to AWS region health.** Outages and throttling events in the AWS kms service translated directly into payment-side errors. A self-hosted key-distribution path decouples this dependency.

The replacement is a self-hosted secrets cluster based on **OpenBao** (the open-source fork of HashiCorp Vault), configured with three properties that together address the three pain points above: *Shamir secret sharing* for the unseal key, *memory-only key material* on the running daemons, and *Raft-based replication* for the storage backend.

4.2 Shamir Secret Sharing for Unseal

The cluster is initialised with a master key K that is never written to durable storage in plaintext. Instead, K is split into n shares using Shamir's (t, n) -threshold scheme so that any t out of n share-holders can reconstruct K , but any $t - 1$ holders learn nothing. In the production deployment we chose $(t, n) = (3, 5)$, so that:

- No single operator can unseal the cluster on their own.

- Up to two share-holders may be unavailable (illness, travel, key-rotation) without halting recovery.
- Compromise of any two shares discloses no information about K .

On every restart of an OpenBao node, the node is in a *sealed* state: the storage backend is encrypted at rest, but the daemon holds no decryption key in memory. Three of the five share-holders submit their shares (over an authenticated TLS channel from their respective workstations); the daemon reconstructs K in memory, decrypts the storage encryption key, and transitions to *unsealed*. At no point is K written to disk on the OpenBao node.

4.3 Memory-Only Key Material

A central design decision is that all reconstructed key material lives only in process memory. Three consequences of this choice:

- **Disk loss is benign.** If a node's local disk is exfiltrated, the attacker obtains only the encrypted storage backend, which is useless without K .
- **Restart requires re-unseal.** This is intentional. Spontaneous reboots (whether from kernel panic, OOM, or scheduled maintenance) leave the cluster sealed until the operators participate; this reduces blast radius from any post-boot compromise.
- **Memory-protection hygiene matters.** The OpenBao process is configured with `mlock(MCL_CURRENT | MCL_FUTURE)` so the unsealed key is never paged to swap. `ptrace` is disabled via `prctl(PR_SET_DUMPABLE, 0)`; core dumps are suppressed; and the systemd unit runs with `ProtectKernelTunables`, `NoNewPrivileges`, and a restrictive `seccomp` profile.

4.4 Raft-Based Storage Replication

The encrypted storage backend is replicated across an odd-numbered Raft cluster (typically 5 voters across 3 availability zones). Raft was chosen over consul-style gossip for two reasons: (i) it gives a hard, well-understood linearisability guarantee for writes, which matches the semantics secrets consumers expect; and (ii) it removes the operational dependency on a separate Consul cluster.

Cluster membership is managed via OpenBao's `raft join` and `raft autopilot` subsystems, which automatically demote and remove voters that fall behind, and promote stable non-voters once they catch up. Failover testing showed automatic leader re-election within 5–7 seconds of a controlled leader kill, with no observable secret-fetch failures from the application side (clients retry on the standard sealed/leader-loss errors).

4.5 Integration into the Application Path

The application-side change was minimal. A new client wraps the OpenBao API and exposes the same Decrypt surface as the previous AWS KMS client. A typical request now proceeds:

1. Application requests a transit-engine decrypt on an envelope-encrypted payload.
2. OpenBao server-side decrypts the data-encryption key (DEK) using the in-memory transit key, returns the DEK over TLS.
3. Application uses the DEK locally to decrypt the payload.
4. DEK is zeroised in process memory immediately after use.

The transit engine is configured with key-rotation enabled and key-version pinning so that re-keying does not break in-flight ciphertexts.

4.6 Outcomes

- **KMS API spend** reduced significantly on the migrated workloads (the precise figure is not reproduced here for commercial reasons).
- **Audit posture** improved: the unseal procedure is now documented, drilled quarterly, and produces an immutable audit trail of which share-holders participated.
- **Decoupling from AWS region health** eliminated a class of cross-region incidents that had historically appeared as payment failures.
- **Latency** for Decrypt on the hot path is comparable to AWS KMS in-region latency ($\sim 2\text{--}4$ ms p99) and substantially better cross-region.
- **Failure modes are explicit:** loss of any two voters or any two share-holders leaves the cluster operational; loss of three triggers an explicit, alarmed degradation rather than silent data loss.

Chapter 5

Barbican Key Management Service with HSM Integration

5.1 Introduction and Motivation

In multi-tenant cloud environments, the secure generation, storage, and lifecycle management of cryptographic secrets (encryption keys, certificates, passwords, and opaque data blobs) is a critical requirement. Regulatory frameworks such as **PCI-DSS**, **HIPAA**, **GDPR**, and **SOC 2** mandate that sensitive key material be protected using tamper-resistant hardware and that tenants' cryptographic assets be strictly isolated from one another.

Barbican is the official OpenStack Key Management Service (KMS), providing a RESTful API for the secure storage, provisioning, and management of secret data. When paired with a **Hardware Security Module (HSM)**, Barbican achieves the highest levels of cryptographic assurance by ensuring that master encryption keys **never leave the HSM boundary** in plaintext.

This chapter describes the architecture, implementation, and integration of Barbican with an HSM to establish a **per-tenant secret management pipeline** where each tenant's secrets are encrypted under a unique, HSM-protected project-level Key Encryption Key (KEK).

5.2 Background Concepts

5.2.1 What Is Barbican?

Barbican is a component of the OpenStack ecosystem that provides:

- **Secret Storage:** Stores symmetric keys, asymmetric key pairs, certificates, passphrases, and arbitrary binary data.
- **Secret Lifecycle Management:** Supports creation, retrieval, deletion, and metadata management for secrets.
- **Access Control:** Integrates with OpenStack Keystone for authentication and uses role-based access control (RBAC) policies to enforce per-project (tenant) permissions.
- **Pluggable Backend Architecture:** Supports multiple secret store backends via a plugin interface, including software-based stores and HSMs.

5.2.2 What Is a Hardware Security Module (HSM)?

An HSM is a dedicated, tamper-resistant cryptographic processor designed to:

- Generate cryptographic keys using a certified true random number generator (TRNG).
- Perform encryption, decryption, signing, and verification operations **within the HSM boundary**, ensuring that key material is never exposed to the host system's memory.
- Provide **FIPS 140-2 Level 3** (or higher) certification, meeting regulatory requirements for key protection.
- Support industry-standard interfaces: **PKCS#11** (Cryptoki), **KMIP**, and vendor-specific APIs.

Examples of HSMs include Thales Luna Network HSM, Entrust nShield, and AWS CloudHSM.

5.2.3 The Envelope Encryption Model

Barbican+HSM uses a two-tier **envelope encryption** model:

1. **Master Key (MK)**: Resides exclusively within the HSM. Never exported. Used to wrap/unwrap project-level KEKs.
2. **Key Encryption Key (KEK)**: One per OpenStack project (tenant). Generated by the HSM, encrypted (wrapped) under the MK, and stored in Barbican's database in wrapped form.
3. **Data Encryption Key (DEK)**: The actual secret being stored (e.g., a volume encryption key). Encrypted under the project's KEK before being persisted to the database.

5.3 Architecture and Design

5.3.1 System Components

The complete system involves the following interacting components:

1. **OpenStack Keystone**: Identity service providing authentication tokens and project (tenant) scoping.
2. **Barbican API Server**: Receives RESTful requests to create, retrieve, list, and delete secrets. Authenticates requests via Keystone middleware.

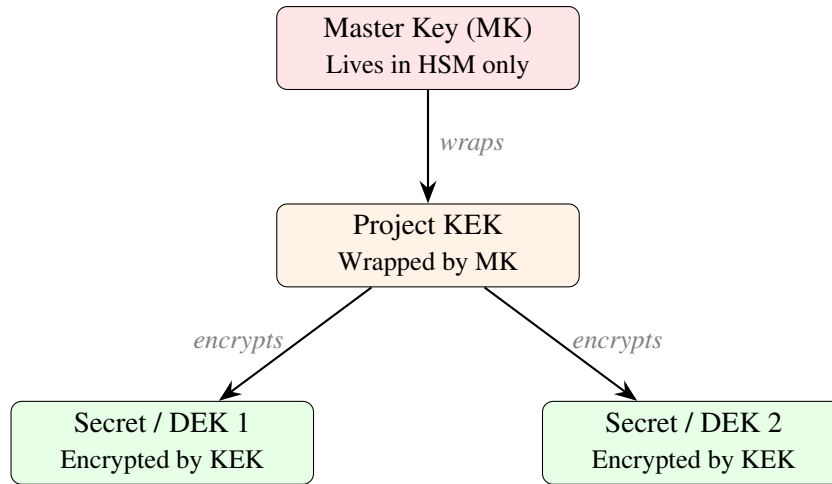


Figure 5.1: Envelope Encryption Hierarchy in Barbican with HSM

3. **Barbican Worker:** Asynchronous task processor handling secret encryption/decryption operations.
4. **Secret Store Plugin (PKCS#11):** The bridge between Barbican and the HSM. Communicates with the HSM via the PKCS#11 interface.
5. **HSM (Hardware Security Module):** Performs all cryptographic operations. Stores the Master Key.
6. **Database (PostgreSQL/MySQL):** Stores encrypted secrets, wrapped KEKs, and metadata.
7. **Message Queue (RabbitMQ):** Facilitates communication between the API server and workers.

5.3.2 Per-Tenant Key Isolation

A critical design requirement is **per-tenant cryptographic isolation**. This is achieved as follows:

1. When a tenant (identified by their OpenStack `project_id`) stores their first secret, Barbican's PKCS#11 plugin triggers the HSM to **generate a new KEK** specifically for that project.
2. The HSM generates the KEK internally, wraps it using the Master Key, and returns the **wrapped KEK** to Barbican.
3. Barbican stores the wrapped KEK in the database, associated with the `project_id` in the `kek_datum` table.

4. For all subsequent secrets from that tenant, Barbican retrieves the wrapped KEK, sends it to the HSM for unwrapping, uses the unwrapped KEK to encrypt the secret (all within the HSM), and stores the encrypted secret.
5. **Result:** Each tenant's secrets are encrypted under a unique key. Compromising one tenant's KEK does not affect other tenants. The Master Key, which protects all KEKs, never leaves the HSM.

5.4 Implementation Details

5.4.1 HSM Configuration

The HSM is initialised with the following parameters:

```

1 # Initialise the HSM partition for Barbican
2 lunacm> partition create -name barbican_partition \
3     -password <partition_password>
4
5 # Generate the Master Key (MKEK) on the HSM
6 lunacm> cmu generatekeypair -keytype AES \
7     -keylength 256 \
8     -label "barbican_hmac_key" \
9     -extractable false
10
11 # Generate the HMAC key for integrity verification
12 lunacm> cmu generatekeypair -keytype SHA256_HMAC \
13     -label "barbican_hmac_key" \
14     -extractable false

```

Listing 5.1: HSM Initialisation (Thales Luna Example)

5.4.2 Barbican Configuration

The Barbican service is configured to use the PKCS#11 secret store plugin:

```

1 [secretstore]
2 enabled_secretstore_plugins = store_crypto
3
4 [crypto]
5 enabled_crypto_plugins = p11_crypto
6
7 [p11_crypto_plugin]
8 # Path to the HSM's PKCS#11 library
9 library_path = /usr/lib/libCryptoki2_64.so
10
11 # HSM login credentials

```

```

12 login = <partition_password>
13
14 # Label of the Master KEK on the HSM
15 mkek_label = barbican_master_kek
16 mkek_length = 32
17
18 # Label of the HMAC key on the HSM
19 hmac_label = barbican_hmac_key
20
21 # HSM slot ID
22 slot_id = 1
23
24 # Token serial number (optional, for multi-slot setups)
25 # token_serial_number = <serial>

```

Listing 5.2: barbican.conf — PKCS#11 Plugin Configuration

5.4.3 Secret Creation Flow

The following sequence describes the end-to-end flow when a tenant creates a secret:

1. API Request:

```

1 curl -X POST https://barbican.example.com/v1/secrets \
2   -H "X-Auth-Token: $TOKEN" \
3   -H "Content-Type: application/json" \
4   -d '{
5     "name": "db-encryption-key",
6     "algorithm": "AES",
7     "bit_length": 256,
8     "mode": "CBC",
9     "secret_type": "symmetric",
10    "payload": "BASE64_ENCODED_KEY_MATERIAL",
11    "payload_content_type": "application/octet-stream",
12    "payload_content_encoding": "base64"
13  }'

```

2. **Authentication:** The Keystone middleware validates the token and extracts the `project_id`.

3. KEK Lookup/Generation:

- Barbican queries the `kek_datum` table for an existing KEK associated with this `project_id`.
- If no KEK exists (first secret for this tenant):
 - (a) Barbican's PKCS#11 plugin calls `C_GenerateKey()` on the HSM to create a new 256-bit AES KEK.

- (b) The HSM wraps the new KEK with the Master Key via `C_WrapKey()`.
- (c) The wrapped KEK is stored in the `kek_datum` table.

- If a KEK already exists, the wrapped KEK is retrieved from the database.

4. Secret Encryption:

- (a) The wrapped KEK is sent to the HSM.
- (b) The HSM unwraps it using the Master Key (`C_UnwrapKey()`).
- (c) The HSM encrypts the secret payload using the unwrapped KEK (`C_Encrypt()`).
- (d) The encrypted secret and an HMAC tag (for integrity) are returned to Barbican.

5. **Persistence:** The encrypted secret, HMAC, algorithm metadata, and a reference to the KEK datum are stored in the Barbican database.

6. **Response:** A 201 Created response with the secret's UUID reference is returned to the tenant.

5.4.4 Secret Retrieval Flow

1. Tenant sends a GET request to `/v1/secrets/{uuid}/payload`.
2. Barbican retrieves the encrypted secret and associated KEK datum from the database.
3. The wrapped KEK and encrypted payload are sent to the HSM.
4. The HSM unwraps the KEK, decrypts the payload, and returns the plaintext secret.
5. The plaintext secret is returned to the tenant over TLS.

5.5 Security Properties

5.6 Challenges and Learnings

1. **PKCS#11 Library Compatibility:** Different HSM vendors implement the PKCS#11 standard with subtle variations. Debugging session management and error codes required close collaboration with vendor documentation.
2. **HSM Session Pooling:** PKCS#11 sessions are a limited resource. Barbican's worker pool had to be tuned to match the HSM's maximum concurrent session count, avoiding session exhaustion under load.

Property	How It Is Achieved
Key Material Protection	Master Key never leaves the HSM; all cryptographic operations occur within HSM boundary
Per-Tenant Isolation	Each project has a unique KEK; secrets from different tenants cannot be decrypted with the wrong KEK
Defence in Depth	Even if the database is compromised, secrets are encrypted and KEKs are wrapped—unusable without HSM access
Integrity Verification	HMAC tags ensure that encrypted secrets have not been tampered with
Audit Trail	All PKCS#11 operations are logged by both the HSM and Barbican, providing a complete audit trail
Regulatory Compliance	FIPS 140-2 Level 3 HSM certification satisfies PCI-DSS, HIPAA, and SOC 2 requirements

Table 5.1: Security Properties Achieved via Barbican and HSM Integration

3. **High Availability:** The HSM was configured in an HA pair with automatic failover. Barbican’s configuration was updated to support multiple HSM addresses for seamless failover.
4. **Performance Considerations:** HSM operations are inherently slower than software-based encryption (due to the hardware interface). Caching unwrapped KEKs in volatile HSM memory for short periods (using PKCS#11 session objects) improved throughput without compromising security.

5.7 Summary

The integration of Barbican with an HSM establishes a production-grade, multi-tenant key management system where cryptographic secrets are generated, encrypted, and stored with the highest levels of assurance. Per-tenant KEK isolation ensures that a compromise in one tenant’s security posture does not cascade to others. The envelope encryption model, combined with HSM-backed key protection, satisfies stringent regulatory requirements while remaining operationally transparent to end users through Barbican’s RESTful API.

Chapter 6

Distributed Event-Driven Payout Scheduling System (Namma Yatri)

6.1 Introduction and Business Context

In ride-hailing platforms, timely and accurate driver payouts are essential for driver satisfaction, retention, and platform trust. Traditionally, driver payouts are processed in **weekly batch cycles**, meaning a driver who completes a ride on Monday may not receive payment until the following Monday—a delay that is particularly burdensome in markets where drivers depend on daily cash flow.

This project addresses this challenge by designing and implementing a **distributed, event-driven payout scheduling system** for special economic zones. The system processes payouts for over **10,000 rides daily**, reducing payout latency from weekly cycles to **T+2 hours**—meaning a driver receives their payout within 2 hours of completing a ride.

6.2 System Requirements

The system was designed to satisfy the following requirements:

1. **Low Latency:** Payout processing must complete within 2 hours of ride completion (T+2 hours).
2. **Exactly-Once Execution:** Each ride payout must be processed exactly once—no duplicates, no missed payments.
3. **Scalability:** The system must handle 10,000+ rides per day without degradation.
4. **Non-Interference:** Payout scheduling must not impact ride-booking latency. The critical path (ride booking) must remain unaffected.
5. **Fault Tolerance:** The system must gracefully handle failures (network partitions, service crashes, payment gateway timeouts) without data loss or inconsistent states.
6. **Idempotency:** All operations must be idempotent, allowing safe retries after transient failures.

7. **Reconciliation:** The system must support automated reconciliation for payment discrepancies.

6.3 Architecture Design

6.3.1 High-Level Architecture

The system follows an **event-driven architecture** with the following components:

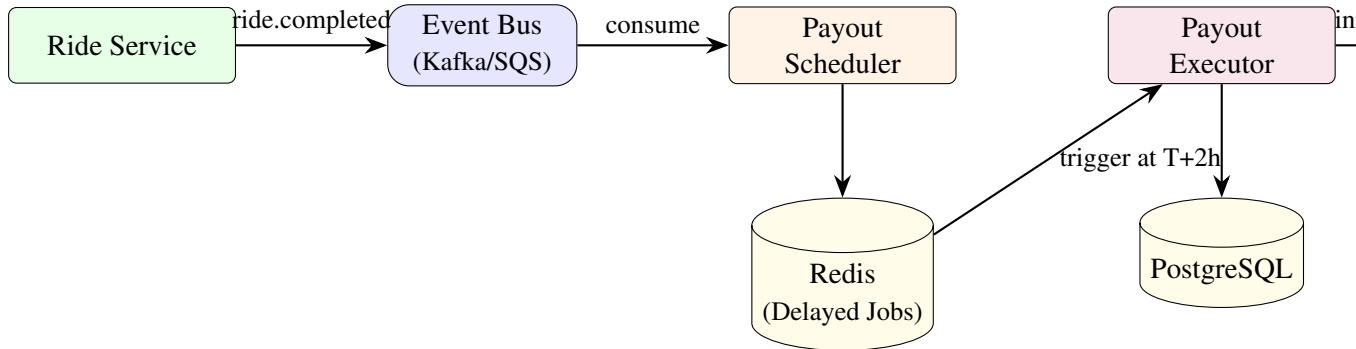


Figure 6.1: High-Level Architecture of the Payout Scheduling System

6.3.2 Component Responsibilities

Ride Service: Emits a `ride.completed` event to the event bus upon ride completion. This is a fire-and-forget operation that adds zero latency to the ride-booking critical path.

Event Bus (Kafka/SQS): Provides durable, ordered message delivery. Serves as a decoupling layer between the ride service and the payout subsystem.

Payout Scheduler: Consumes `ride.completed` events and schedules a delayed payout job in Redis with a 2-hour delay. Applies per-ride locking to prevent duplicate scheduling.

Redis (Delayed Job Store): Stores scheduled payout jobs using sorted sets (ZADD) with the execution timestamp as the score. Provides $O(1)$ scheduling and $O(\log N)$ retrieval of due jobs.

Payout Executor: A worker process that polls Redis for jobs whose execution timestamp has passed. Processes payouts with exactly-once semantics using idempotent state transitions.

Payment Gateway (Paytm EDC): The external payment provider. Handles the actual fund transfer to the driver's account.

PostgreSQL: Persistent storage for payout records, state transitions, and audit logs.

6.4 Detailed Design

6.4.1 Redis-Backed Delayed Job Scheduler

The delayed job scheduler is the core innovation enabling the T+2-hour payout guarantee. It leverages Redis sorted sets for efficient scheduling:

```

1 import redis
2 import time
3 import json
4 import hashlib
5
6 r = redis.Redis(host='redis-cluster', port=6379, db=0)
7
8 PAYOUT_QUEUE = "payout:delayed_jobs"
9 PAYOUT_LOCK_PREFIX = "payout:lock:"
10 PAYOUT_DELAY_SECONDS = 7200 # 2 hours
11
12 def schedule_payout(ride_id, driver_id, amount, zone_id):
13     """
14     Schedule a payout job with T+2 hour delay.
15     Uses per-ride locking for exactly-once scheduling.
16     """
17     lock_key = f"{PAYOUT_LOCK_PREFIX}{ride_id}"
18
19     # Acquire per-ride lock (NX = set if not exists, EX = TTL)
20     if not r.set(lock_key, "locked", nx=True, ex=86400):
21         # Lock already exists => payout already scheduled
22         return False # Idempotent: no duplicate scheduling
23
24     execution_time = time.time() + PAYOUT_DELAY_SECONDS
25     job = json.dumps({
26         "ride_id": ride_id,
27         "driver_id": driver_id,
28         "amount": amount,
29         "zone_id": zone_id,
30         "scheduled_at": time.time(),
31         "idempotency_key": hashlib.sha256(
32             f"{ride_id}:{driver_id}".encode()
33         ).hexdigest()
34     })
35
36     # ZADD with score = execution timestamp
37     r.zadd(PAYOUT_QUEUE, {job: execution_time})
38     return True

```

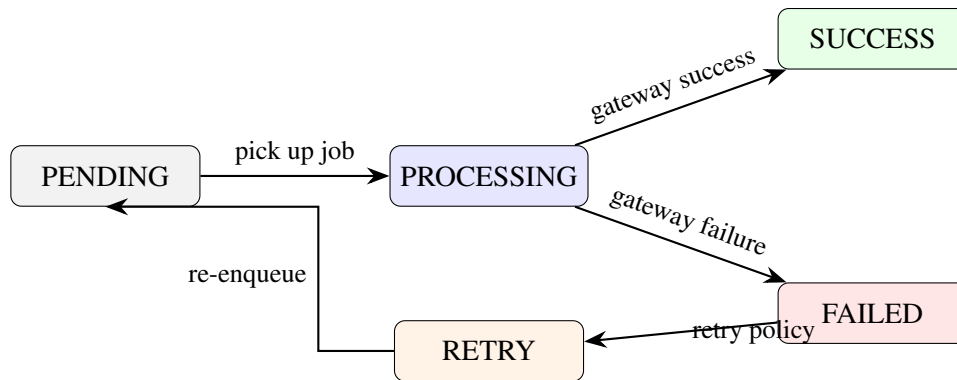
Listing 6.1: Delayed Job Scheduling Logic

Key design decisions:

- **Per-Ride Locking (SET NX):** Ensures that even if the `ride.completed` event is delivered multiple times (at-least-once delivery), the payout is scheduled only once. The lock has a 24-hour TTL for automatic cleanup.
- **O(1) Scheduling:** ZADD is an $O(\log N)$ operation on the sorted set, but since jobs are append-mostly, the amortised cost is effectively $O(1)$.
- **Idempotency Key:** A deterministic hash of `ride_id` and `driver_id` serves as a deduplication key at the payment gateway level.

6.4.2 Payout Executor with Idempotent State Management

The executor processes due jobs with a carefully designed state machine:



```

1 def execute_due_payouts():
2     """
3     Poll for jobs whose execution time has passed.
4     Process with exactly-once semantics.
5     """
6     now = time.time()
7
8     # ZRANGEBYSCORE: get all jobs with score <= now
9     due_jobs = r.zrangebyscore(
10         PAYOUT_QUEUE, '-inf', now, start=0, num=100
11     )
12
13     for job_data in due_jobs:
14         job = json.loads(job_data)
15         ride_id = job['ride_id']
16
17         # Begin state transition: PENDING -> PROCESSING
18         if not transition_state(ride_id, 'PENDING', 'PROCESSING'):
19             continue # Already processing or completed
20

```

```

21     try:
22         # Call payment gateway with idempotency key
23         result = initiate_payout(
24             driver_id=job['driver_id'],
25             amount=job['amount'],
26             idempotency_key=job['idempotency_key']
27         )
28
29         if result.success:
30             transition_state(ride_id, 'PROCESSING', 'SUCCESS')
31             r.zrem(PAYOUT_QUEUE, job_data) # Remove from queue
32         else:
33             handle_failure(job, result)
34
35     except Exception as e:
36         transition_state(ride_id, 'PROCESSING', 'FAILED')
37         schedule_retry(job, retry_count=job.get('retries', 0))
38
39 def transition_state(ride_id, from_state, to_state):
40     """
41     Atomic state transition using database CAS
42     (Compare-And-Swap) semantics.
43     """
44     result = db.execute("""
45         UPDATE payout_jobs
46         SET state = %s, updated_at = NOW()
47         WHERE ride_id = %s AND state = %s
48     """, (to_state, ride_id, from_state))
49     return result.rowcount == 1 # True if transition succeeded

```

Listing 6.2: Payout Executor with State Machine

6.4.3 Paytm EDC Integration

The system integrates with **Paytm EDC (Electronic Data Capture)** terminals—Android-based Point-of-Sale (POS) devices—for processing in-zone payments. The integration involves:

Type-Safe API Design

All API interactions with the Paytm EDC gateway are modelled using strongly-typed request/response structures:

```

1 from dataclasses import dataclass
2 from enum import Enum
3 from typing import Optional

```

```

4 from decimal import Decimal
5
6 class PaymentStatus(Enum):
7     INITIATED = "INITIATED"
8     SUCCESS = "SUCCESS"
9     FAILED = "FAILED"
10    REFUNDED = "REFUNDED"
11    PENDING = "PENDING"
12
13 @dataclass(frozen=True)
14 class PayoutRequest:
15     ride_id: str
16     driver_id: str
17     amount: Decimal
18     currency: str = "INR"
19     idempotency_key: str = ""
20     zone_id: str = ""
21
22 @dataclass(frozen=True)
23 class PayoutResponse:
24     transaction_id: str
25     status: PaymentStatus
26     amount: Decimal
27     timestamp: str
28     error_code: Optional[str] = None
29     error_message: Optional[str] = None

```

Listing 6.3: Type-Safe Payment API Models

Reconciliation Engine

Automated reconciliation ensures consistency between the platform's records and the payment gateway's records:

```

1 def reconcile_payouts(date):
2     """
3     Daily reconciliation between internal records
4     and gateway settlement reports.
5     """
6     internal_records = db.query("""
7         SELECT ride_id, amount, status, transaction_id
8         FROM payout_jobs
9         WHERE DATE(created_at) = %s AND status = 'SUCCESS'
10    """, (date,))
11
12    gateway_records = paytm_api.get_settlement_report(date)
13

```

```

14     # Build lookup maps
15     internal_map = {r.transaction_id: r for r in internal_records}
16     gateway_map = {r.txn_id: r for r in gateway_records}
17
18     discrepancies = []
19
20     # Check for records in our DB but not in gateway
21     for txn_id, record in internal_map.items():
22         if txn_id not in gateway_map:
23             discrepancies.append({
24                 'type': 'MISSING_AT_GATEWAY',
25                 'ride_id': record.ride_id,
26                 'amount': record.amount
27             })
28
29     # Check for amount mismatches
30     for txn_id in internal_map.keys() & gateway_map.keys():
31         if internal_map[txn_id].amount != gateway_map[txn_id].amount:
32             discrepancies.append({
33                 'type': 'AMOUNT_MISMATCH',
34                 'ride_id': internal_map[txn_id].ride_id,
35                 'internal_amount': internal_map[txn_id].amount,
36                 'gateway_amount': gateway_map[txn_id].amount
37             })
38
39     return discrepancies

```

Listing 6.4: Reconciliation Logic

Refund Handling

For failed transactions where the payment was debited but the payout was not completed, the system implements automated refund processing:

```

1 def handle_failed_transaction(job, gateway_response):
2     """
3     If the gateway reports a debit but the payout failed,
4     initiate a refund.
5     """
6     if (gateway_response.status == PaymentStatus.FAILED and
7         gateway_response.error_code == "DEBIT_SUCCESS_CREDIT_FAIL
8         "):
9
10         refund_result = paytm_api.initiate_refund(
11             original_txn_id=gateway_response.transaction_id,
12             amount=job['amount'],
13             reason="CREDIT_FAILURE_AUTO_REFUND"

```

```

13         )
14
15     db.execute("""
16         INSERT INTO refund_log
17         (ride_id, original_txn_id, refund_txn_id,
18          amount, status, created_at)
19         VALUES (%s, %s, %s, %s, %s, NOW())
20     """, (job['ride_id'],
21          gateway_response.transaction_id,
22          refund_result.refund_id,
23          job['amount'],
24          refund_result.status))

```

Listing 6.5: Refund Handling for Failed Transactions

Status Polling

To handle asynchronous payment processing, the system implements robust status polling with exponential backoff:

```

1 import time
2
3 def poll_payment_status(transaction_id, max_attempts=10):
4     """
5     Poll the payment gateway for final transaction status.
6     Uses exponential backoff to avoid overwhelming the gateway.
7     """
8     base_delay = 1 # seconds
9     for attempt in range(max_attempts):
10         response = paytm_api.check_status(transaction_id)
11
12         if response.status in (PaymentStatus.SUCCESS,
13                               PaymentStatus.FAILED):
14             return response # Terminal state reached
15
16         # Exponential backoff: 1s, 2s, 4s, 8s, ...
17         delay = base_delay * (2 ** attempt)
18         delay = min(delay, 60) # Cap at 60 seconds
19         time.sleep(delay)
20
21     # Max attempts exhausted; mark for manual review
22     return PayoutResponse(
23         transaction_id=transaction_id,
24         status=PaymentStatus.PENDING,
25         amount=Decimal('0'),
26         timestamp=str(time.time()),
27         error_message="Status polling timed out"

```

28

)

Listing 6.6: Payment Status Polling with Backoff

6.5 Performance Characteristics

Metric	Value
Daily Ride Volume	10,000+ rides
Payout Latency	T+2 hours (previously weekly)
Scheduling Throughput	O(1) per ride (Redis ZADD)
Exactly-Once Guarantee	Per-ride locks + idempotent state machine
Impact on Ride Booking Latency	Zero (async event-driven)
Retry Policy	Exponential backoff, max 5 retries
Reconciliation Frequency	Daily automated + weekly manual audit

Table 6.1: Performance Characteristics of the Distributed Payout System

6.6 Challenges and Learnings

1. **Redis Persistence vs. Performance:** Redis' in-memory nature provides speed but risks data loss. The system uses Redis with AOF (Append-Only File) persistence and periodic RDB snapshots. Additionally, all job metadata is persisted to PostgreSQL before being enqueued in Redis, providing a recovery path.
2. **Clock Skew in Distributed Systems:** The delayed job mechanism relies on timestamps. In a multi-node deployment, clock skew between nodes could cause premature or delayed job execution. This was mitigated by using NTP synchronisation and accepting a $\pm 500\text{ms}$ tolerance window.
3. **Payment Gateway Idempotency:** Not all payment gateways natively support idempotency keys. For Paytm EDC, the idempotency contract was verified empirically, and the system was designed to handle non-idempotent edge cases via reconciliation.
4. **Handling Zone-Specific Regulations:** Special economic zones have different tax and regulatory requirements. The system parameterises payout calculations based on zone configuration, allowing per-zone customisation without code changes.
5. **Monitoring and Alerting:** Custom metrics were added:
 - `payout.scheduled.count` — Number of payouts scheduled per minute
 - `payout.executed.count` — Number of payouts executed per minute

- `payout.failed.count` — Number of failed payouts (triggers alert if > threshold)
- `payout.latency.p99` — 99th percentile payout processing time

6.7 Summary

The distributed payout scheduling system transforms driver payments from a slow, batch-oriented process into a near-real-time, event-driven pipeline. Key achievements include:

- **97%+ reduction in payout latency** (weekly → 2 hours)
- **Exactly-once payment processing** via per-ride locking and idempotent state machines
- **Zero impact on ride-booking critical path** through asynchronous event-driven architecture
- **Automated reconciliation and refund handling** eliminating manual intervention for payment discrepancies
- **Production-proven at scale** serving 10,000+ rides daily

Chapter 7

Self-Directed Project: PostgreSQL on Kubernetes/OpenStack

7.1 Motivation and Scope

Industry teams routinely operate Postgres on Kubernetes on top of OpenStack-managed VMs in production, but the full stack — from bare metal up through the database — is rarely available to a single engineer to exercise end-to-end. This self-directed project replicated that stack on a single workstation (an Intel i9-14900) with sufficient fidelity that it could be used as both a learning vehicle and a reproducible testbed for failure-injection experiments.

The project produced three engineering deliverables:

1. A five-layer nested virtualisation stack tuned for write-heavy database workloads.
2. A zero-touch, end-to-end automation pipeline that brings a fresh OpenStack tenant from empty to a running, highly-available PostgreSQL cluster in roughly 12 minutes.
3. A Zero-Trust secret delivery framework with a Go-based broker and a Python SDK that VMs use to fetch their secrets at first boot without long-lived credentials.

7.2 Five-Layer Nested Virtualisation Stack

The stack consists of, from the bottom up:

Layer 1 — Bare metal: Intel i9-14900, 24 cores (8 P-cores + 16 E-cores), 128 GB DDR5, NVMe storage.

Layer 2 — KVM host: Linux + libvirt with VT-x and EPT enabled; nested virtualisation toggled on (`kvm-intel.nested=1`).

Layer 3 — OpenStack control plane: A single-host OpenStack deployment (Nova, Neutron, Cinder, Glance, Keystone, Barbican) running inside a single privileged VM. Storage backed by Cinder over Ceph; networking via OVN.

Layer 4 — Kubernetes: HA Kubernetes cluster (3 control-plane + 3 worker nodes) provisioned as OpenStack instances, control plane behind a kube-vip virtual IP.

Layer 5 — PostgreSQL: CloudNativePG-managed Postgres cluster (1 primary + 2 synchronous replicas) running on the K8s layer, with PVCs backed by Cinder (and, transitively, by the Layer 1 NVMe).

7.2.1 Performance Tuning

Naive nested virtualisation is slow: every database write traverses five virtual block devices and four network stacks. Two tuning passes recovered the bulk of the loss:

- **NUMA-aware CPU pinning.** The 8 P-cores were partitioned across the OpenStack control VM, the K8s control-plane VMs, and the K8s workers. Each level was given an exclusive, NUMA-pinned set of vCPUs via libvirt's `<cputune>` and OpenStack's `hw:cpu_policy=dedicated`. Cross-NUMA traffic was eliminated for the database workers.
- **16 GB hugepages.** Both the OpenStack VMs and the database PVCs were backed by 1 GB hugepages, totalling 16 GB pinned. This reduced TLB pressure visibly and stabilised `pgbench` TPS variance.

The combined effect on a TPC-B-like `pgbench` workload was an approximately **50 %** increase in write TPS over the un-tuned baseline.

7.3 Zero-Touch Cluster Provisioning

The objective was that running a single command (`make cluster`) should yield a verified-healthy HA Postgres cluster, with no operator interaction beyond entering an SSH key.

The pipeline composes three open-source projects:

Cluster API for OpenStack (CAPO): declarative provisioning of K8s clusters as OpenStack workloads. CAPO consumes a `Cluster`, `OpenStackCluster`, and `KubeadmControlPlane` manifest and reconciles the live state of the OpenStack tenant against them.

CloudNativePG (CNPG): a Kubernetes operator that provisions and manages PostgreSQL clusters as first-class K8s resources. The operator handles streaming replication, synchronous-replica election, automatic failover, and PITR via base backups + WAL archive.

A small Make/shell wrapper: orchestrates the OpenStack tenant bootstrap, applies the CAPI manifests, waits for cluster readiness, then applies the CNPG manifests.

7.3.1 Time Breakdown

End-to-end provisioning of an empty OpenStack tenant to a healthy 3-replica Postgres cluster takes roughly **12 minutes** on the i9-14900:

- OpenStack tenant bootstrap (network, security groups, image): ~1 min
- Control-plane VM creation + cloud-init: ~3 min

- Worker VM creation + cloud-init: ~3 min
- kubeadm + CNI bring-up: ~2 min
- CNPG operator install + Postgres cluster creation: ~3 min

7.3.2 Failover Behaviour

A controlled `kill -9` of the Postgres primary results in CNPG promoting a synchronous replica within ~7 seconds, with **RPO = 0** confirmed by replaying the test workload against the new primary and observing no missing transactions. The same drill against an entire K8s control-plane node (simulated power loss of one of the three) similarly recovered within the kube-vip + etcd liveness budget without data loss.

7.4 Zero-Trust Secret Delivery Framework

The provisioning pipeline must hand secrets (database passwords, certificate keys, OAuth client credentials) to fresh VMs without requiring those VMs to ship pre-baked credentials. The constraint is the well-known *secret-zero* bootstrapping problem.

7.4.1 Architecture

A small Go-based broker, `osvault-broker`, sits adjacent to the OpenBao cluster (Chapter 4). The flow is:

1. A fresh VM, on first boot, calls the OpenStack **Instance Metadata Service (IMDS)** for a signed identity document containing `instance_id`, `project_id`, and a Nova-signed assertion of liveness.
2. The VM sends this assertion to `osvault-broker` over the local OpenStack tenant network.
3. The broker validates the IMDS signature, looks up the instance's role in OpenBao's policy DB, and mints a short-lived (5 min) **RS256-signed JWT** carrying the appropriate OpenBao token.
4. The VM presents the JWT to OpenBao and receives its scoped secrets.

No long-lived secret material is ever placed in cloud-init userdata or in the VM image. The trust chain rests on the IMDS signing key (rotated weekly) and the broker's RSA key (held only in OpenBao's transit engine).

7.4.2 Custom Python SDK: osvault

To make consumption ergonomic, a small Python SDK named `osvault` wraps the broker handshake. A typical first-boot script looks like:

```
1 from osvault import VaultClient
2
3 client = VaultClient.from_imds()           # auto-discovers IMDS,
      fetches RS256 JWT
4 db = client.get("postgres/credentials")   # short-lived DB password
5 tls = client.get("tls/server-cert")       # private key + cert chain
6
7 write_pgpass(db.username, db.password)
8 write_tls_pair(tls.cert, tls.key)
```

Listing 7.1: First-boot secret retrieval via osvault

The SDK handles JWT renewal, retries on transient broker unavailability, and clears all secret material from memory after use (`ctypes.memset` on the underlying buffers).

7.5 Outcomes and Reusability

The stack is reproducible from a single Git repository and a Make target. It has served as a sandbox for several follow-on experiments: chaos-style failure injection on the Postgres layer, latency profiling of nested networking, and dry-runs of OpenStack upgrades against a representative tenant. Beyond personal learning, the resulting Make pipeline and the `osvault` SDK are reusable artifacts: the pipeline has been adopted internally as the canonical way to spin up disposable test clusters, and the SDK has been packaged as an internal Python wheel.

Chapter 8

Conclusion

8.1 Summary

This internship year exposed me to three distinct strata of the production distributed-systems stack: *block storage* (Lightbits NVMe/TCP replacing CEPH at Juspay), *in-process state management* (the two-tier caching layer for high-traffic payment APIs), and *secrets infrastructure* (the Shamir-secret-sharing-based OpenBao cluster, with the supporting Barbican KMS + HSM integration). In parallel, the open-source contribution to Namma Yatri delivered a distributed event-driven payout scheduler under live production traffic, and the self-directed PostgreSQL on Kubernetes/OpenStack project closed the loop by exercising the full stack from bare metal to database on a single workstation.

The headline numbers across these projects — $4\times$ storage efficiency on Lightbits, 4.1 % cross-node call reduction and 11.2 % p95 latency reduction on caching, 7-second automatic failover on the K8s/Postgres stack with RPO=0, and the migration of Namma Yatri driver payouts from weekly cycles to T+2 hours — are individually meaningful but, taken together, point to a recurring engineering principle: the largest wins came not from new abstractions but from *taking correctness seriously while doing more with fewer machines*.

8.2 Skills Developed

- **Production distributed-systems engineering:** writing code that runs on real money-moving traffic; understanding tail latency, blast radius, and recovery as first-class design constraints.
- **Cross-stack debugging:** reasoning about a single transaction's path across application, cache, Redis, OpenStack volume, and Lightbits storage — and being able to attribute latency to any layer.
- **Open-source collaboration:** the Namma Yatri PR went through public review with the community and a maintainer who set a high bar for design clarity, idempotency, and test coverage.
- **Cryptographic engineering hygiene:** Shamir secret sharing, RS256 JWT discipline, mlock-based memory protection, and the operational drills that go with self-custodial key material.

- **Systems-level performance tuning:** NUMA-aware CPU pinning, hugepages, nested virtualisation, and the empirical method of validating each tuning step against a reproducible benchmark.

8.3 Future Directions

- Extend the caching framework with a contention layer that handles single-resource state safely; this is the subject of the parallel B.Tech project report.
- Generalise the OpenBao broker into a small, reusable open-source project (currently internal-only) so other teams running OpenStack can adopt the pattern.
- Run the PostgreSQL on K8s/OpenStack pipeline as a chaos-engineering testbed: scripted failure injection at each of the five virtualisation layers, with end-to-end correctness invariants.

Bibliography

- [1] Lightbits Labs, “LightOS Technical Documentation,” <https://www.lightbitlabs.com/resources/>
- [2] Weil, S.A., et al., “Ceph: A Scalable, High-Performance Distributed File System,” *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
- [3] NVM Express, Inc., “NVMe over Fabrics Specification,” <https://nvmexpress.org/specifications/>
- [4] OpenStack Barbican, “Key Manager Service Documentation,” <https://docs.openstack.org/barbican/latest/>
- [5] RSA Laboratories, “PKCS #11: Cryptographic Token Interface Standard,” <https://www.cryptsoft.com/pkcs11doc/>
- [6] NIST, “FIPS 140-2: Security Requirements for Cryptographic Modules,” 2001.
- [7] Redis Ltd., “Redis Sorted Sets Documentation,” <https://redis.io/docs/data-types/sorted-sets/>
- [8] Apache Software Foundation, “Apache Kafka Documentation,” <https://kafka.apache.org/documentation/>
- [9] Paytm, “EDC Payment Gateway API Documentation,” <https://developer.paytm.com/>
- [10] Helland, P., “Idempotence Is Not a Medical Condition,” *Communications of the ACM*, vol. 55, no. 5, 2012.
- [11] Shamir, A., “How to Share a Secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [12] Ongaro, D. and Ousterhout, J., “In Search of an Understandable Consensus Algorithm (Raft),” *Proceedings of the 2014 USENIX Annual Technical Conference*, 2014.
- [13] OpenBao Maintainers, “OpenBao: Open Source Secrets Management,” <https://openbao.org/>, 2025.
- [14] Kubernetes SIGs, “Cluster API and Cluster API Provider OpenStack (CAPO),” <https://cluster-api.sigs.k8s.io/>, 2025.

- [15] EnterpriseDB, “CloudNativePG: Postgres Operator for Kubernetes,” <https://cloudnative-pg.io/>, 2025.
- [16] Kumar, A., et al., “Comparative Analysis of Distributed Caching Algorithms,” arXiv:2504.02220, 2024.
- [17] Namma Yatri Project, “Open-Source Mobility Platform,” <https://github.com/nammayatri/nammayatri>, 2026.